

Volatility-Adjusted Memory Protection

From a synthetic win to a validated control knob — with negative controls

Richard · 2026 · Revised draft (v2)

This revision supersedes the original 3-page portfolio note (archived as `volatility_ewc_portfolio_v1_original.pdf`). It reconciles the write-up with the Split-MNIST, capacity, library, and multi-turn results and their negative controls. Source: `paper/build_paper.py`; companion: `paper/findings.md`.

TL;DR — Standard continual-learning methods (EWC) protect all past knowledge with one fixed strength, forcing a tradeoff between remembering old tasks and adapting to new ones. I scale protection by how volatile each task's domain actually is — measured *before* any protection is applied. On a synthetic benchmark this improved both axes at once (+2.7pp stable retention, +5.2pp volatile adaptation). On real data (Split-MNIST) the honest result is subtler: it is *not* a free-lunch Pareto win but a validated **control knob** that causally steers the stability-plasticity tradeoff — confirmed by a negative control (shuffling or inverting the domain→volatility map removes or reverses the effect). The same principle drives a working LLM-memory library whose behaviour is likewise volatility-driven under control.

1. The Problem

Continual-learning systems face the *stability-plasticity dilemma*: protect old knowledge too strongly and the model cannot adapt; too weakly and it forgets. Elastic Weight Consolidation (EWC) penalises changes to parameters in proportion to their past importance (Fisher information). EWC works, but applies one global protection strength uniformly — every past task is guarded equally regardless of whether its domain is still relevant. Not all knowledge ages the same way: core language structure barely shifts; a user's current preferences might change monthly. Treating both equally wastes protection budget on memories going stale and starves memories that actually deserve to last.

2. Hypothesis

Scale the EWC penalty per stored task by how volatile that task's domain actually is, measured before any protection is applied. High measured volatility → weak protection; low volatility → strong protection.

$$w_d = 1 / v_d^\gamma$$

The original prediction was that this improves both retention and adaptability *simultaneously*. That turned out to be true only in a specific regime; the general, real-data claim is weaker and more precise (Section 5).

3. Method

Synthetic benchmark

12 sequential binary classification tasks on 2D Gaussian blobs, alternating between two domain types sharing a single trunk and single output head — genuine weight competition, no dedicated capacity to hide in. **Stable domain**: blob centres jitter ± 0.15 (boundary barely moves). **Volatile domain**: blob centres relocate substantially each task.

Real data — Split-MNIST

To answer the obvious objection (“2D blobs are a toy”), the same idea runs on Split-MNIST. A stable pool of digits keeps a fixed digit→class mapping every task; a disjoint volatile pool is randomly re-partitioned each task so successive volatile tasks contradict each other. Both feed one shared trunk and one 2-way head, so they genuinely compete for weights (real forgetting), but because the pools are disjoint, protecting stable features does not directly fight volatile inputs.

Volatility estimation (pre-update) & penalty

Before any gradient step on a new task, measure the model's loss using only knowledge from prior tasks — a clean mismatch signal uncorrupted by the EWC penalty — and track a running EMA per domain. Each stored task's EWC penalty is then weighted by w_d above: high $V_d \rightarrow$ small $w_d \rightarrow$ weak protection; low $V_d \rightarrow$ large $w_d \rightarrow$ strong protection.

Negative control (the load-bearing part)

Every non-synthetic claim is gated on a **sabotage / negative control**: re-run the identical pipeline with the domain $\rightarrow$ volatility mapping (i) shuffled and (ii) inverted. A genuine effect must degrade monotonically **REAL** > **SHUFFLE** > **SWAP**. If a “win” survives shuffling or inversion, it is generic non-uniform regularisation, not the volatility signal, and it is discarded. Several early wins failed this test — that filter is why the surviving claims are trustworthy.

4. Failures Worth Keeping

Early attempts produced flat ties or artifacts — kept because diagnosing them was most of the actual work:

- **Separate output heads** — no genuine weight competition, nothing to forget in either method.
- **Volatility measured post-update** — contaminated by EWC clamping; it tracked the intervention, not the domain.
- **Models trained to 100% confidence** — collapses Fisher information toward zero, making the penalty inert. Fixed by label smoothing and fewer epochs.
- **High-capacity MNIST “win”** (hidden=512, +7pp retention) — *failed the negative control*: shuffling/inverting the pairing kept most of the gain, so it was generic regularisation, not volatility. Discarded as a headline result.
- **Capacity-saving hypothesis** (a small volatility model matching a larger baseline) — *falsified* by the sweep (Section 5.3).

5. Results

5.1 Synthetic — both axes improve (regime-specific)

Metric	Baseline EWC	Volatility-Adjusted
Early-task stable retention	0.935	0.962 (+2.7pp)
Late-task volatile adaptation	0.576	0.628 (+5.2pp)
Avg accuracy — stable tasks	0.936	0.963 (+2.7pp)
Avg accuracy — volatile tasks	0.617	0.633 (+1.6pp)

8 runs, 12 tasks each. The model inferred the volatile domain was $\sim 2.4\times$ more unstable than the stable one, purely from pre-update loss. Holds when tasks use disjoint inputs over a shared trunk; not a universal Pareto win (see 5.2).

5.2 Split-MNIST — a causal control knob

In a genuinely-competing regime (hidden=128, $\lambda \approx 300$, 16 tasks, 6 runs) the effect *passes* the negative control. Reported as a stability index = retention – adaptation:

	baseline	REAL	SHUFFLE	SWAP
stability index	0.317	0.460	0.457	0.405

$REAL - SWAP = +0.055$ in the predicted direction: inverting which domains are “volatile” reliably tilts the model toward plasticity. The volatility signal causally steers the tradeoff — but it trades adaptation for retention along a frontier a well-tuned uniform baseline could also reach.

5.3 Capacity — robustness to under-tuned protection

The parameter-savings hypothesis was falsified. The favourable, causal benefit appears in **under-protected large models**, where uniform EWC catastrophically forgets and volatility rescues the stable domain (hidden=256, $\lambda=300$, 4 runs):

- retention: baseline 0.877 $\rightarrow$ **0.954** (+7.7pp); adaptation only -2.9 pp,
- passes the sabotage control (shuffle/swap keep only +1.3pp).

Interpretation: volatility auto-allocates a fixed protection budget to stable knowledge, reducing the need to hand-tune λ per capacity. A well-tuned uniform baseline can match it; volatility mainly reduces tuning burden.

5.4 LLM-memory library — volatility-driven under control

The same principle powers a small persistent-memory library. Running it end-to-end under three volatility profiles — real (priors), flat (all equal), swap (inverted):

Capability	real	flat	swap
Selective updating (accuracy)	100%	83%	50%
Freshness-aware retrieval (separation)	+0.589	+0.202	-0.267

The swap control degrading below flat is the causal evidence: flipping which domains are volatile flips behaviour in the wrong direction.

5.5 Reliability-weighted volatility EMA

A multi-turn robustness fix: the volatility EMA's learning rate is scaled by source reliability, so $\alpha = (1 - \beta) \cdot \text{clamp}(R_t, 0, 1)$ and $V \leftarrow (1 - \alpha)V + \alpha M_t$. A low-trust observation moves a stable memory **2.5× less** than the prior logic; reliable-source updates are unchanged (backward compatible). A sustained weak stream crosses $V \geq 0.5$ at turn 8 versus the old logic's turn 2, and stable content is never wrongly overwritten. (Also fixed: observe() previously updated the EMA twice per call.) Honest limit: reliability scales the step size, not the EMA's fixed point, so persistent contradictions still raise volatility over time — arguably correct, but not a hard “never drifts” guarantee.

5.6 LLM-agent memory — beating naive policies on both failure modes

As memory for an LLM agent, the same principle is tested over many noisy sessions where facts drift at domain-appropriate rates. A memory layer can fail two ways at once: go *stale* on volatile facts, or get *corrupted* by a confident-but-wrong observation on stable facts. Naive policies are forced onto one side; VoltMem escapes the tradeoff (24 sessions $\times$ 20 runs; balanced = harmonic mean of stable & volatile accuracy):

system	overall	stable	volatile	balanced
voltmem (real)	0.522	0.578	0.617	0.597
voltmem (flat)	0.361	0.578	0.188	0.283
voltmem (swap)	0.347	0.407	0.194	0.263
always-overwrite	0.573	0.360	0.767	0.490
never-overwrite	0.361	0.578	0.188	0.283
reliability-threshold	0.579	0.437	0.548	0.486

VoltMem-real is the only policy strong on both axes; each naive policy catastrophically fails one, and a pure reliability rule is mediocre on both. real > flat > swap is the causal control. Building the demo also surfaced a real bug: observe() updated the volatility EMA before the escalation decision, so one confident blip inflated volatility and lowered its own threshold, overwriting a stable fact on first contact. Fixed by deciding against the volatility known before the observation, then learning from it.

5.7 Slot-aware linking — write-path wedge vs Mem0

Embedding-based *remember()* must associate a new turn with an existing memory before *observe()* can apply the escalation rule. A single global similarity threshold fails on paraphrases (MiniLM scores 0.44–0.53 on

mood/preference rephrasings). **Slot-aware linking** adds a domain-scoped fallback: volatile singleton slots (mood, location, task) and preference sibling domains link at a volatility-scaled lower threshold.

scenario	VoltMem	Mem0 (real)
mood update	1 fact, correct top	2 facts, stale top
preference change	1 fact, correct top	2 facts, stale top
location change	1 fact, correct top	2 facts, stale top

Scripted case study (3/3 current-truth wins), not a public-benchmark SOTA claim. Linking quality depends on the embedding backend.

5.8 Retrieval haystack & LongMemEval-S

Noisy haystack (5 slots $\times$ 20 runs, 6 stale decoys + 3 distractors per slot, top-5):

system	current@1	current@5	stale@1	separation
voltmem_real	0.600	1.000	0.000	0.153
voltmem_flat	0.800	1.000	0.200	0.133
voltmem_swap	0.650	1.000	0.200	0.111
similarity_only	0.600	0.800	0.200	−0.003

PASS: real avoids stale volatile traps (0% stale@1 vs 20% cosine), finds current in top-5 more often (100% vs 80%), separation real > swap. On LongMemEval-S (n=60, chunk-calibrated ingest), answer@5 is 0.700 (real) tying cosine; flat 0.717 leads slightly; swap 0.667 < real. Preference type recovers to 0.700 (was 0.300 with heuristic domains).

6. Caveats

- **Not a free-lunch accuracy method.** On real data it is a control knob / tuning-robustness tool, not a universal Pareto improvement.
- **Regime-dependent.** The “improve both axes” result needs disjoint task inputs over a shared trunk; benefits are strongest in under-protected large models.
- **Small-scale benchmarks only.** 2D blobs and Split-MNIST; Split-CIFAR, larger nets, and longer task streams are pending.
- **EMA fixed-point drift.** Reliability weighting slows erosion but does not anchor the estimate to the domain prior; a prior-anchored update is the natural next step.
- **Embedding backend variance.** Linking thresholds are calibrated for sentence-transformers (MiniLM); production should pin or calibrate per backend.
- **LongMemEval overall.** At n=30, answer@5 does not beat plain cosine; report per-type causal signals honestly.
- **Manual partitioning.** Automatic volatility detection from gradient-conflict signals remains open work.
- **Vector index (v0.2).** SQLite ANN accelerates candidate retrieval; volatility re-rank is unchanged (engineering, not a separate claim).
- **No replay-baseline comparison** yet (GEM, A-GEM, Synaptic Intelligence) — volatility-weighting may be additive or redundant.

7. Why This Is on My Portfolio

This started as a philosophical question — when should a person trust an old habit vs. question it — that turned out to have a direct mechanistic analogue in a real ML problem. The path from idea to equations to working code to null result to diagnosed bug to real result — and then to *discarding my own apparent wins when they failed a negative control* — is most of what applied ML engineering is day-to-day. Arriving, as a self-taught engineer, at a result that maps onto a genuine open problem in continual-learning research, and being disciplined enough to state

precisely what it is and is not, is the actual thing this piece demonstrates.

*Code: github.com/Rouche01/voltmem · Package: pypi.org/project/voltmem (v0.2.0) · Reproduction: [paper/findings.md](#),
[docs/RESEARCH.md](#)*